

NAME_____ ROLL NO._____ DATE_____

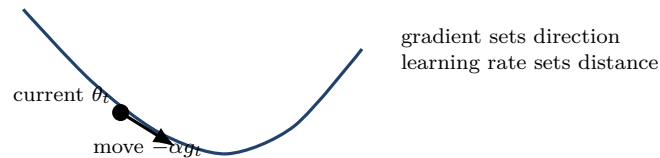
[illegible]

Part 0 The baseline: SGD reacts to one estimate

0.1 Start with the step every optimiser modifies

For parameters θ , a gradient $g_t = \nabla_{\theta} L(\theta_t)$ points in the direction of increasing loss. Gradient descent walks in the opposite direction:

Rule $\theta_{t+1} = \theta_t - \alpha g_t$.



The limitation. For stochastic or mini-batch gradient descent, g_t is an estimate from a small sample. One batch may point slightly uphill, sideways, or against a longer trend.

Analogy. SGD is like steering a boat by looking only at the newest ripple. The optimisers below keep a small memory: not the full history, just enough state to make the next steering decision less fragile.

Prediction

With $\theta_t = 2$, $\alpha = .1$, $g_t = 3$, does θ increase or decrease? What information would vanilla SGD retain for the next step?

Part 1 The first memory: EWMA

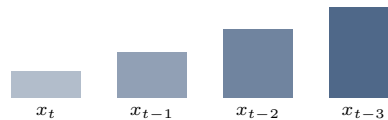
1.1 Exponentially weighted moving average

An **exponentially weighted moving average** (EWMA) keeps a fading summary of a sequence x_t :

Rule $s_t = \beta s_{t-1} + (1 - \beta)x_t, \quad 0 \leq \beta < 1.$

Analogy. Imagine a coach judging a player's form. Yesterday matters most, last week still matters, and last month matters a little. The coach does not keep every observation in hand; one running impression, s_t , is enough.

older observations receive another factor of β



The newest observation has weight $1 - \beta$; older observations fade geometrically.

1.2 Unroll the rule

Substitute $s_{t-1} = \beta s_{t-2} + (1 - \beta)x_{t-1}$:

$$s_t = \beta^2 s_{t-2} + \beta(1 - \beta)x_{t-1} + (1 - \beta)x_t.$$

Read it. A large β gives a longer memory but a slower reaction. A small β reacts faster but smooths less.

Given In the next activity, use $s_0 = 0$, $\beta = .8$, $x_1 = 5$, $x_2 = 1$.

Question before calculation

Which sample should influence s_2 more: $x_1 = 5$ or $x_2 = 1$? State the coefficient that supports your claim.

Part 1 EWMA calculation ladder

1.3 Build the fading memory

Given $s_0 = 0$, $\beta = .8$, $x_1 = 5$, $x_2 = 1$. Use $s_t = .8s_{t-1} + .2x_t$.

Step 1. Name the initial memory.

$$s_0 = \mathbf{0}.$$

Step 2. Update with the first observation.

$$s_1 = .8(\mathbf{0}) + .2(\mathbf{5}) = \mathbf{1.0}.$$

Step 3. Carry that state into the second update.

$$s_2 = .8(\mathbf{1.0}) + .2(\mathbf{1}) = \mathbf{1.0}.$$

Step 4. Direction check.

The newer x_2 has coefficient **.2**; the older x_1 has coefficient **.16**. Which is larger?

newer influence: **.2** older influence: **.16**.

EWMA does not store all history. It stores one state, s_t .

Practice: make a new memory

Given $s_0 = 0$, $\beta = .6$, $x_1 = 4$, $x_2 = 2$.

Step 1. Calculate s_1 .

$$s_1 = .6(\mathbf{0}) + .4(\mathbf{4}) = \mathbf{1.6}.$$

Step 2. Calculate s_2 .

$$s_2 = .6(\mathbf{1.6}) + .4(\mathbf{2}) = \mathbf{1.76}.$$

Step 3. Explain in one line why s_2 is not simply the average of 4 and 2.

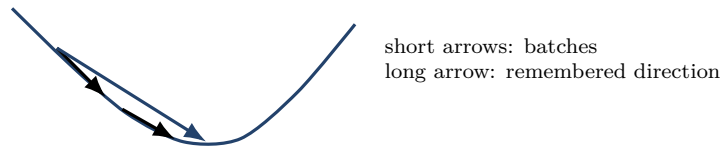
Part 2 Momentum uses remembered direction

2.1 SGD with Momentum

Momentum turns the EWMA of gradients into a velocity:

Rule $v_t = \beta v_{t-1} + (1 - \beta)g_t, \quad \theta_{t+1} = \theta_t - \alpha v_t.$

Problem it solves. A narrow valley can give gradients that alternate across the valley even while the useful downhill direction stays similar. Plain SGD can zig-zag; Momentum averages the zig-zag into a steadier travel direction.



A rolling bicycle does not instantly face a new direction after one small wobble.

2.2 What is remembered?

Current gradient g_t the newest slope signal
Velocity v_t the fading direction history
Learning rate α how far to move along the velocity

Given $v_0 = 0, \theta_0 = 2.00, \beta = .9, \alpha = .1, g_1 = 4, g_2 = -2.$

Prediction. Will the second velocity v_2 be positive, negative, or zero? Justify without calculating.

Part 2 Momentum calculation ladder

2.3 Update state first, parameter second

Given $v_0 = 0$, $\theta_0 = 2.00$, $\beta = .9$, $\alpha = .1$, $g_1 = 4$, $g_2 = -2$.

Step 1. Old velocity contribution.

$$\beta v_0 = .9(\mathbf{0}) = \mathbf{0}.$$

Step 2. New gradient contribution.

$$(1 - \beta)g_1 = .1(\mathbf{4}) = \mathbf{.4}.$$

Step 3. Combine to obtain the first velocity.

$$v_1 = \mathbf{0} + \mathbf{.4} = \mathbf{.4}.$$

Step 4. Use velocity, not raw gradient, in the parameter rule.

$$\theta_1 = 2.00 - .1(\mathbf{.4}) = \mathbf{1.96}.$$

Step 5. Repeat with the opposing second gradient.

$$v_2 = .9(\mathbf{.4}) + .1(-2) = \mathbf{.16}.$$

$$\theta_2 = \mathbf{1.96} - .1(\mathbf{.16}) = \mathbf{1.944}.$$

Step 6. Direction check.

Although g_2 is negative, the remembered velocity is **positive**. Therefore θ still **decreases**, but more slowly.

Momentum remembers a direction, not a list of old gradients.

Practice

For $g_3 = -2$, calculate v_3 and θ_3 from $v_2 = .16$, $\theta_2 = 1.944$.

$$v_3 = .9(\mathbf{.16}) + .1(-2) = \mathbf{-.056}, \quad \theta_3 = 1.944 - .1(\mathbf{-.056}) = \mathbf{1.9496}.$$

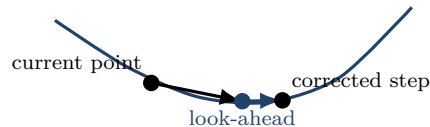
Part 3 NAG checks the place it is heading

3.1 Nesterov accelerated gradient

Nesterov Accelerated Gradient (NAG) uses momentum to predict a location, measures the gradient there, and then corrects the velocity:

Rule $\theta_{\text{look}} = \theta_t - \alpha \beta v_t, \quad v_{t+1} = \beta v_t + (1 - \beta) \nabla L(\theta_{\text{look}}), \quad \theta_{t+1} = \theta_t - \alpha v_{t+1}.$

Analogy. A cyclist approaching a sharp bend looks where the bicycle is about to be, not only at the wheel's current position. If the road rises at the bend, they can brake before overshooting it.



3.2 The order matters

1. Predict θ_{look} using the old velocity.
2. Measure ∇L at θ_{look} , not at θ_t .
3. Correct velocity, then update the actual parameter.

Given $\theta_t = 2.000, v_t = .500, \alpha = .1, \beta = .9, \nabla L(\theta_{\text{look}}) = 3.000.$

Prediction. Which labelled point is used to calculate the NAG gradient? Explain why it is not the current point.

Part 3 NAG calculation ladder

3.3 Predict, measure, correct, move

Given $\theta_t = 2.000$, $v_t = .500$, $\alpha = .1$, $\beta = .9$, $\nabla L(\theta_{\text{look}}) = 3.000$.

Step 1. Predict the look-ahead parameter.

$$\theta_{\text{look}} = 2.000 - .1(.9)(.500) = \mathbf{1.955}.$$

Step 2. Write the two terms that form the corrected velocity.

$$\beta v_t = .9(.500) = \mathbf{.450}, \quad (1 - \beta)\nabla L(\theta_{\text{look}}) = .1(3.000) = \mathbf{.300}.$$

Step 3. Add those terms.

$$v_{t+1} = \mathbf{.450} + \mathbf{.300} = \mathbf{.750}.$$

Step 4. Use the corrected velocity in the real update.

$$\theta_{t+1} = 2.000 - .1(\mathbf{.750}) = \mathbf{1.925}.$$

Step 5. Direction check.

The current velocity moved the look-ahead point **downhill**; the gradient sampled there **corrected** the next velocity.

Momentum asks: what is the slope here? NAG asks: what is the slope where momentum is taking me?

Practice

Given $\theta_t = 1.500$, $v_t = .200$, $\alpha = .1$, $\beta = .8$, $\nabla L(\theta_{\text{look}}) = -1.000$.

$$\theta_{\text{look}} = 1.500 - .1(.8)(.200) = \mathbf{1.484}, \quad v_{t+1} = .8(\mathbf{.200}) + .2(-1) = \mathbf{-.040}, \quad \theta_{t+1} = 1.500 - .1(\mathbf{-.040}) = \mathbf{1.504}.$$

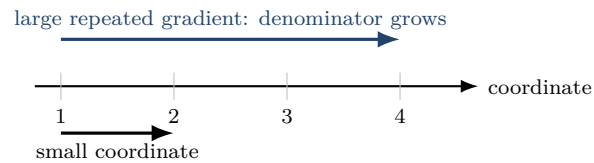
Part 4 AdaGrad counts coordinate scale

4.1 Different coordinates can need different step sizes

AdaGrad stores a cumulative squared-gradient state for each coordinate:

Rule $r_t = r_{t-1} + g_t^2, \quad \theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{r_t} + \varepsilon}.$

Analogy. A hiker uses shorter steps on loose scree and regular steps on firm ground. AdaGrad gives each coordinate its own cautiousness meter: repeatedly large gradients create a larger denominator and a shorter effective step.



The square removes sign when measuring scale, but the original g_t remains in the direction numerator.

Given $r_{t-1} = (0, 0)$, $g_t = (4, 1)$, and ignore ε .

Question before calculation

Will the first coordinate retain its raw size 4 after normalisation? Why or why not?

Part 4 AdaGrad calculation ladder

4.2 Accumulate, take square roots, normalise

Given $r_{t-1} = (0, 0)$, $g_t = (4, 1)$, and ignore ε .

Step 1. Square each gradient coordinate.

$$g_t^2 = (4^2, 1^2) = (\mathbf{16}, \mathbf{1}).$$

Step 2. Update the accumulator coordinate by coordinate.

$$r_t = (0, 0) + (\mathbf{16}, \mathbf{1}) = (\mathbf{16}, \mathbf{1}).$$

Step 3. Take the square-root scale.

$$\sqrt{r_t} = (\sqrt{\mathbf{16}}, \sqrt{\mathbf{1}}) = (\mathbf{4}, \mathbf{1}).$$

Step 4. Divide the raw gradient by its own scale.

$$\frac{g_t}{\sqrt{r_t}} = \left(\frac{4}{\mathbf{4}}, \frac{1}{\mathbf{1}} \right) = (\mathbf{1}, \mathbf{1}).$$

Step 5. Interpret.

The coordinate with raw gradient 4 takes an effective direction of **1**; the coordinate with raw gradient 1 also takes **1**.

AdaGrad makes repeated high-scale coordinates cautious. Its drawback: the cumulative denominator never forgets.

Practice: the next identical gradient

Given $r_t = (16, 1)$, $g_{t+1} = (4, 1)$, and ignore ε .

$$r_{t+1} = (\mathbf{32}, \mathbf{2}), \quad g_{t+1}/\sqrt{r_{t+1}} = (\mathbf{.707}, \mathbf{.707}).$$

Part 4 RMSProp forgets old scale

4.3 Replace permanent accumulation with fading scale

RMSProp uses an EWMA of squared gradients:

$$\textbf{Rule} \quad s_t = \rho s_{t-1} + (1 - \rho) g_t^2, \quad \theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{s_t} + \varepsilon}.$$

Analogy. AdaGrad keeps every old weather report when deciding whether to take an umbrella. RMSProp remembers the recent climate and lets a storm from months ago fade away.

	AdaGrad	RMSProp
State	all old g^2	fading g^2 average
Old scale	stays forever	is repeatedly discounted
Late training risk	increasingly timid	scale can recover

Given $s_{t-1} = (16, 1)$, $g_t = (4, 1)$, $\rho = .9$, and ignore ε .

Prediction

In this first RMSProp step, s_t happens to equal $(16, 1)$. Does that mean RMSProp is identical to AdaGrad? Explain using what happens to a very old squared gradient.

Part 4 RMSProp calculation ladder

4.4 Update the fading scale

Given $s_{t-1} = (16, 1)$, $g_t = (4, 1)$, $\rho = .9$, and ignore ε .

Step 1. Square the incoming gradient.

$$g_t^2 = (4^2, 1^2) = (\mathbf{16}, \mathbf{1}).$$

Step 2. Keep ninety percent of the old scale.

$$\rho s_{t-1} = .9(16, 1) = (\mathbf{14.4}, \mathbf{.9}).$$

Step 3. Add ten percent of the new squared gradient.

$$(1 - \rho)g_t^2 = .1(16, 1) = (\mathbf{1.6}, \mathbf{.1}).$$

$$s_t = (\mathbf{14.4}, \mathbf{.9}) + (\mathbf{1.6}, \mathbf{.1}) = (\mathbf{16}, \mathbf{1}).$$

Step 4. Normalise the gradient.

$$g_t / \sqrt{s_t} = (4 / \sqrt{\mathbf{16}}, 1 / \sqrt{\mathbf{1}}) = (\mathbf{1}, \mathbf{1}).$$

Practice: changed recent history

Given $s_{t-1} = (4, 9)$, $g_t = (2, 3)$, $\rho = .5$, and ignore ε .

$$s_t = .5(4, 9) + .5(4, 9) = (\mathbf{4}, \mathbf{9}), \quad g_t / \sqrt{s_t} = (\mathbf{1}, \mathbf{1}).$$

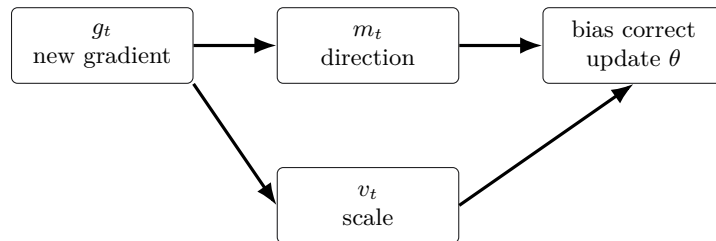
Step 5. Explain why this method can recover from an old high-gradient period whereas AdaGrad cannot.

Part 5 Adam keeps two memories

5.1 Direction memory plus scale memory

Adam combines Momentum and RMSProp:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$



Rule $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$

Why hats? Starting $m_0 = v_0 = 0$ pulls early moving averages towards zero. Bias correction removes this artificial start-up effect.

Given $m_0 = 0, v_0 = 0, g_1 = 4, \beta_1 = .9, \beta_2 = .999, \alpha = .01.$ Ignore ε .

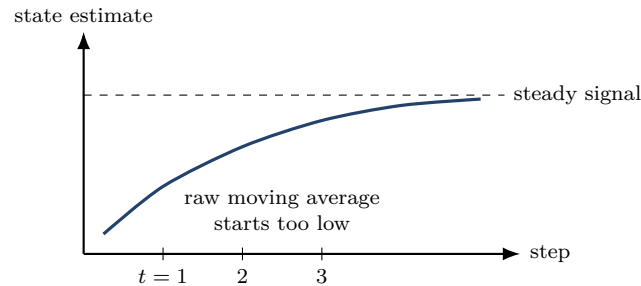
Question before calculation

Which state should retain the sign of the gradient: m_t or v_t ? Why?

Part 5 Why Adam corrects its early memories

5.2 A zero start makes the first estimates too small

Both m_t and v_t begin at zero. That is convenient, but it causes an artificial early bias: before enough gradients have arrived, each moving average contains too much zero.



Analogy. If a class average begins with several imaginary zero marks, the first real mark looks too small. Bias correction removes the imaginary zero marks:

Rule $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$

Prediction

At $t = 1$, $\beta_1 = .9$, and raw $m_1 = .4$. Is dividing by $1 - .9$ going to make the estimate larger or smaller? Why is that the desired correction?

Part 5 Adam calculation ladder

5.3 First moment, second moment, then correction

Given $m_0 = 0, v_0 = 0, g_1 = 4, \beta_1 = .9, \beta_2 = .999, \alpha = .01$. Ignore ε .

Step 1. Calculate the first moment, the direction memory.

$$m_1 = .9(0) + .1(4) = .4.$$

Step 2. Calculate the second moment, the scale memory.

$$v_1 = .999(0) + .001(4^2) = .016.$$

Step 3. Bias-correct the first moment.

$$\hat{m}_1 = \frac{.4}{1 - .9} = 4.$$

Step 4. Bias-correct the second moment.

$$\hat{v}_1 = \frac{.016}{1 - .999} = 16.$$

Step 5. Form the normalised direction.

$$\hat{m}_1 / \sqrt{\hat{v}_1} = 4 / \sqrt{16} = 1.$$

Step 6. Calculate the parameter movement.

$$\Delta\theta_1 = .01(1) = .01, \quad \theta_1 = \theta_0 - .01.$$

Adam's m_t supplies a remembered direction. Its v_t supplies an adaptive scale.

Short continuation

For $m_1 = .4, v_1 = .016, g_2 = 2$, write the next two raw state updates:

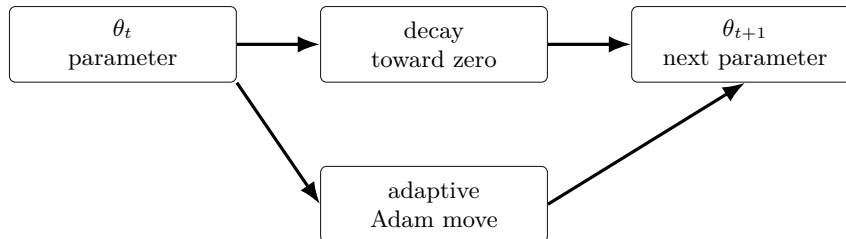
$$m_2 = .9(.4) + .1(2) = .56, \quad v_2 = .999(.016) + .001(2^2) = .019984.$$

Part 6 AdamW: shrink independently of adaptive scale

6.1 Why separate the two jobs?

Adam decides a coordinate-wise training direction. Weight decay has a different job: it steadily prefers smaller parameters. AdamW makes that separation explicit rather than allowing adaptive scaling to alter the strength of regularisation.

Rule $\theta_{t+1} = (1 - \alpha\lambda)\theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$



Analogy. Steering a bicycle toward the destination and keeping the bicycle close to the centre of a lane are separate controls. AdamW keeps the adaptive steering and the shrinkage control separate.

Prediction

If the normalised Adam direction is negative, does $-\alpha(\text{negative})$ increase or decrease θ ? Does decay still point toward zero?

Part 6 AdamW calculation ladder

6.2 Decoupled weight decay

Keep the two operations separate on paper: decay first, adaptive move second.

Rule $\theta_{t+1} = (1 - \alpha\lambda)\theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$

$$(1 - \alpha\lambda)\theta_t$$

decay term: shrinks the parameter itself

$$\alpha \hat{m}_t / \sqrt{\hat{v}_t}$$

adaptive term: follows Adam's normalised direction

Given $\theta_t = 1.00, \alpha = .01, \lambda = .1$, normalised Adam direction = .50.

Step 1. Calculate the decay-only parameter.

$$(1 - \alpha\lambda)\theta_t = (1 - .01 \cdot .1)(1) = \textbf{.999}.$$

Step 2. Calculate the adaptive move.

$$\alpha(.50) = \textbf{.005}.$$

Step 3. Combine the two operations.

$$\theta_{t+1} = \textbf{.999} - \textbf{.005} = \textbf{.994}.$$

Direction check

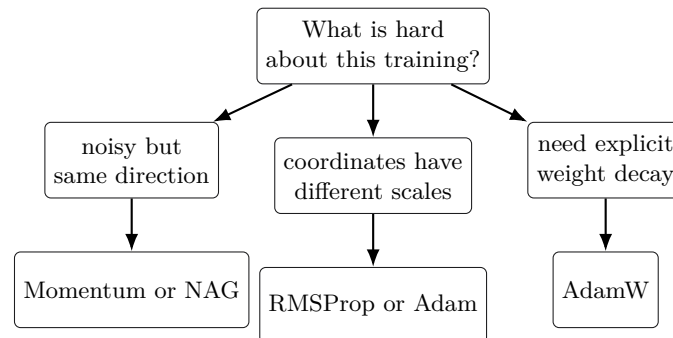
If the normalised Adam direction is negative, the adaptive term $-\alpha(\text{negative})$ moves θ **upward**. Weight decay still shrinks θ toward zero.

Choose the state

EWMA	fading memory of one quantity
Momentum	fading direction history
NAG	direction plus a look-ahead measurement
AdaGrad	cumulative squared-gradient scale
RMSProp	fading squared-gradient scale
Adam	direction and scale memories
AdamW	Adam plus separate parameter shrinkage

Part 7 Choose the right kind of memory

7.1 Optimiser decision map



7.2 Read the diagnoses

Noisy direction	smooth direction with Momentum
Overshoot risk	sample gradient at a prediction with NAG
Uneven coordinates	use per-coordinate scale with AdaGrad or RMSProp
Adaptive baseline	combine both memories with Adam
Adaptive plus decay	use AdamW

Important. These are starting hypotheses, not universal rules. An optimiser is a memory design: choose the memory that addresses the training difficulty you can observe.

Scenario

A model has noisy mini-batch gradients, layers with very different scales, and a need for explicit weight decay. Recommend a starting optimiser and justify it using two states or update terms.

Part 8 Practice and retrieval

8.1 Match the following

- | | | |
|-------------|----------|---|
| 1. AdaGrad | C | A. gradient measured at a predicted point |
| 2. RMSProp | B | B. fading squared-gradient scale |
| 3. Momentum | D | C. squared gradients accumulated forever |
| 4. NAG | A | D. smoothed direction carried forward |
| 5. AdamW | E | E. separate weight decay |

8.2 MCQs

- Increasing EWMA β gives: A. shorter memory B. smoother memory C. no state D. larger raw gradient. **B**
- Momentum stores: A. labels B. a gradient average C. Hessian D. parameter count. **B**
- NAG measures its gradient: A. at the current point B. at look-ahead C. after decay D. nowhere. **B**
- AdaGrad's denominator can grow forever because it uses: A. EWMA B. cumulative g^2 C. g only D. loss. **B**
- RMSProp differs by using: A. fading g^2 scale B. no square C. no learning rate D. labels. **A**
- Adam's signed direction state is: A. v_t B. m_t C. ε D. λ . **B**
- AdamW decouples: A. data and labels B. decay and adaptive update C. two batches D. epoch and step. **B**

Evidence check

Choose any two MCQs. For each, write the question number, your chosen option, and the exact state, update rule, or analogy that makes it true. A correct letter without a reason is only a guess.

Part 8 Explain and transfer

8.3 Subjective questions

Q1. Unroll EWMA one more step and state the coefficient of x_{t-2} .

Answer. $s_t = \beta^3 s_{t-3} + \beta^2(1 - \beta)x_{t-2} + \beta(1 - \beta)x_{t-1} + (1 - \beta)x_t$, so the coefficient is $\beta^2(1 - \beta)$.

Q2. Explain why an opposing gradient may not immediately reverse Momentum.

Answer. The velocity is a weighted average of past gradients. If its existing direction is large, one opposite gradient may reduce it without changing its sign.

Q3. Write NAG's four ordered actions in words.

Answer. Predict a look-ahead point; measure its gradient there; use it to correct the velocity; update the real parameter with that velocity.

Q4. Contrast AdaGrad and RMSProp using their squared-gradient histories.

Answer. AdaGrad sums all past squared gradients, so its denominator only grows. RMSProp keeps a fading EWMA, so old scale can lose influence and the effective step can recover.

Q5. Explain why AdamW's decay is separate from Adam's normalised direction.

Answer. Decoupling makes the shrinkage act directly on the parameter, independently of Adam's coordinate-wise adaptive scale; the adaptive move still follows the normalised gradient direction.